

Guía de DDD, JavaScript y Vue

<https://gemini.google.com/app/34719646e7e8bbed>

User prompt: Dime como hacer esto, con DDD, JavaScript y Vue

Response: Guía paso a paso para desarrollar la solución cumpliendo al 100% con la rúbrica de evaluación, las restricciones de arquitectura Domain-Driven Design (DDD), patrones de diseño (Resource, Assembler, Service) y convenciones de nomenclatura exigidas por la cátedra.

1. Inicialización y Dependencias

Crea el proyecto con Vite indicando el formato de nombre solicitado (pc1 + NRC + código):

```
npm create vite@latest pc14381uXXXXXXX -- --template vue
cd pc14381uXXXXXXX
npm install
```

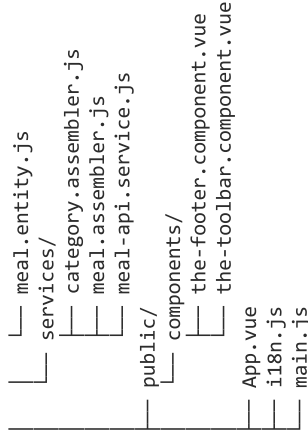
Instala las librerías necesarias: Axios para HTTP, Vue-i18n para internacionalización, y PrimeVue junto con PrimeIcons y PrimeFlex para los estilos e interfaz Material:

```
npm install axios vue-i18n primevue@3.50.0 primeicons primeflex
```

2. Estructura del Proyecto (DDD y Layered Architecture)

La arquitectura divide la solución en los subdominios public y meal, separando responsabilidades en components, services y model:

```
src/
├── assets/
├── meal/
│   ├── components/
│   │   ├── category-list.component.vue
│   │   └── meal-list.component.vue
│   ├── model/
│   │   └── category.entity.js
```



3. Configuración Base

`src/i18n.js`

Configura las traducciones para inglés (por defecto) y español según el enunciado:

```

/**
 * @summary Internationalization configuration for English and Spæ
 * @author Juan Sebastian Estupiñan
 */
import { createI18n } from 'vue-i18n';

const messages = {
  en: {
    title: 'Welcome to MealAll',
    description: 'Here you can find dishes from all over the world',
    footer: 'This page was created by Juan Sanchez.',
    categories: 'Categories',
    meals: 'Dishes',
    selectCategoryPrompt: 'Select a category to view its dishes.'
  },
  es: {
    title: 'Bienvenido Mealall',
    description: 'Aquí podrá encontrar platillos de diferentes de',
    footer: 'Esta pagina fue realizada por Juan Sanchez.',
  }
}

```

```

    categories: 'Categorías',
    meals: 'Platillos',
    selectCategoryPrompt: 'Seleccione una categoría para ver sus f
  }
};

export const i18n = createI18n({
  legacy: false,
  locale: 'en',
  fallbackLocale: 'en',
  messages
});

```

src/main.js

Registra PrimeVue con el prefijo pv- en kebab-case y carga los estilos de Material, PrimeIcons y PrimeFlex:

```

/**
 * @summary Main entry point configuring PrimeVue, i18n, and globa
 * @author Juan Sebastian Estupiñan
 */
import { createApp } from 'vue';
import App from './App.vue';
import { i18n } from './i18n';

import PrimeVue from 'primevue/config';
import Button from 'primevue/button';
import Toolbar from 'primevue/toolbar';
import Card from 'primevue/card';
import Panel from 'primevue/panel';

// PrimeVue Material Theme, Icons & PrimeFlex
import 'primevue/resources/themes/md-light-indigo/theme.css';
import 'primevue/resources/primevue.min.css';
import 'primeicons/primeicons.css';
import 'primeflex/primeflex.css';

const app = createApp(App);

```

```
app.use(PrimeVue, { ripple: true });
app.use(i18n);

// Componentes PrimeVue con prefijo pv-
app.component('pv-button', Button);
app.component('pv-toolbar', Toolbar);
app.component('pv-card', Card);
app.component('pv-panel', Panel);

app.mount('#app');
```

4. Subdominio meal: Entidades, Assemblers y Servicio

src/meal/model/category.entity.js

Representa el recurso de Categoría (Patrón Resource):

```
/**
 * @summary Domain entity representing a Meal Category.
 * @author Juan Sebastian Estupiñan
 */
export class Category {
  constructor({ id, name, thumbnail, description }) {
    this.id = id;
    this.name = name;
    this.thumbnail = thumbnail;
    this.description = description;
  }
}
```

src/meal/model/meal.entity.js

Representa el recurso de Platillo (Patrón Resource):

```
/**
 * @summary Domain entity representing a Dish / Meal.
 * @author Juan Sebastian Estupiñan
 */
```

```
export class Meal {
  constructor({ id, title, thumbnail }) {
    this.id = id;
    this.title = title;
    this.thumbnail = thumbnail;
  }
}
```

src/meal/services/category.assembler.js

Transforma la respuesta cruda del API a entidades de dominio (Patrón Assembler):

```
/**
 * @summary Assembler transforming Category API raw responses into
 * @author Juan Sebastian Estupiñán
 */
import { Category } from '../model/category.entity';

export class CategoryAssembler {
  static toEntityFromResource(resource) {
    return new Category({
      id: resource.idCategory,
      name: resource.strCategory,
      thumbnail: resource.strCategoryThumb,
      description: resource.strCategoryDescription
    });
  }
}

static toEntitiesFromResponse(categories) {
  if (!Array.isArray(categories)) return [];
  return categories.map((cat) => this.toEntityFromResource(cat));
}
```

src/meal/services/meal.assembler.js

Transforma la respuesta cruda de platillos a entidades de dominio (Patrón Assembler):

```

/**
 * @summary Assembler transforming Meal API raw responses into Meal
 * @author Juan Sebastian Estupiñan
 */
import { Meal } from '../model/meal.entity';

export class MealAssembler {
  static toEntityFromResource(resource) {
    return new Meal({
      id: resource.idMeal,
      title: resource.strMeal,
      thumbnail: resource.strMealThumb
    });
  }

  static toEntitiesFromResponse(meals) {
    if (!Array.isArray(meals)) return [];
    return meals.map((meal) => this.toEntityFromResource(meal));
  }
}

```

src/meal/services/meal-api.service.js

Servicio de comunicación HTTP con Axios hacia TheMealDB:

```

/**
 * @summary Service handling HTTP communication with TheMealDB API
 * @author Juan Sebastian Estupiñan
 */
import axios from 'axios';
import { CategoryAssembler } from './category.assembler';
import { MealAssembler } from './meal.assembler';

const http = axios.create({
  baseURL: 'https://www.themealdb.com/api/json/v1/1'
});

export class MealApiService {
  async getCategories() {
    const response = await http.get('/categories.php');
  }
}

```

```

    return CategoryAssembler.toEntitiesFromResponse(response.data);
  }

  async getMealsByCategory(categoryName) {
    const response = await http.get(`/filter.php?c=${categoryName}`)
    return MealAssembler.toEntitiesFromResponse(response.data.meal)
  }
}

```

5. Subdominio public: Toolbar y Footer

`src/public/components/the-toolbar.component.vue`

Toolbar con botones accesibles (ARIA) para alternar el idioma:

```

<template>
  <header role="banner">
    <pv-toolbar class="surface-ground border-none shadow-2 px-4 py">
      <template #start>
        <div class="flex align-items-center gap-2">
          <i class="pi pi-compass text-2xl text-primary" aria-hidden="">
            <span class="text-xl font-bold text-900">MealAll</span>
          </div>
        </template>

        <template #end>
          <div class="flex gap-2" role="group" aria-label="Language">
            <pv-button
              label="EN"
              :severity="locale === 'en' ? 'primary' : 'secondary'"
              size="small"
              aria-label="Change language to English"
              @click="setLocale('en')"
            />
            <pv-button
              label="ES"
              :severity="locale === 'es' ? 'primary' : 'secondary'"
              size="small"
              aria-label="Cambiar idioma a Español"
            />
          </div>
        </template>
      </template>
    </pv-toolbar>
  </header>
</template>

```

```
@click="setLocale('es')"  
>  
  </div>  
</template>  
</pv-toolbar>  
</header>  
</template>  
<script>  
/**  
 * @summary Application top navigation toolbar including i18n lang  
 * @author Juan Sebastian Estupiñan  
 */  
import { useI18n } from 'vue-i18n';  
  
export default {  
  name: 'TheToolbar',  
  setup() {  
    const { locale } = useI18n();  
  
    const setLocale = (lang) => {  
      locale.value = lang;  
    };  
  
    return { locale, setLocale };  
  }  
};  
</script>
```

src/public/components/the-footer.component.vue

Pie de página con soporte para el texto internacionalizado:

```
<template>  
  <footer role="contentinfo" class="surface-ground border-top-1 st  
    <div class="text-center text-600 text-sm">  
      <p class="m-0">{{ $( 'footer' ) }}</p>  
    </div>  
  </footer>  
</template>
```

```

<script>
/**
 * @summary Global application footer component.
 * @author Juan Sebastian Estupiñan
 */
export default {
  name: 'TheFooter',
};
</script>

```

6. Subdominio meal: Vistas y Componentes

`src/meal/components/category-list.component.vue`

Panel izquierdo con lista interactiva de categorías:

```

<template>
  <nav aria-label="Category navigation">
    <div class="flex flex-column gap-2" role="list">
      <div
        v-for="category in categories"
        :key="category.id"
        role="listitem"
        tabindex="0"
        :aria-selected="selectedCategory === category.name"
        :class="[
          'p-3 border-round cursor-pointer flex align-items-center
            selectedCategory === category.name
            ? 'surface-primary text-primary-contrast'
            : 'surface-card hover:surface-hover text-700',
          ]"
        @click="$emit('select-category', category.name)"
        @keydown.enter="$emit('select-category', category.name)"
      >
        
      <span class="font-medium text-lg">{{ category.name }}</span>
    </div>
  </nav>
</template>

<script>
/**
 * @summary Sidebar listing food categories with thumbnail and sel
 * @author Juan Sebastian Estupiñan
 */
export default {
  name: 'CategoryList',
  props: {
    categories: {
      type: Array,
      required: true
    },
    selectedCategory: {
      type: String,
      default: ''
    }
  },
  emits: ['select-category']
};
</script>

```

src/meal/components/meal-list.component.vue

Panel derecho que muestra los platos correspondientes en pv-card:

```

<template>
  <section role="region" :aria-label="$t('meals')">
    <div v-if="meals.length === 0" class="text-center p-5 surface-
      <i class="pi pi-info-circle text-4xl text-400 mb-3" aria-hic
        <p class="text-600 m-0">{{ $t('selectCategoryPrompt') }}</p>
    </div>
  </section>

```

```

<div v-else class="grid">
  <div
    v-for="dish in meals"
    :key="dish.id"
    class="col-12 sm:col-6 lg:col-4 p-2"
  >
    <pv-card class="h-full shadow-2 overflow-hidden flex flex-
      <template #header>
        
      </template>
    <template #title>
      <span class="text-lg font-semibold line-height-2 block"
        {{ dish.title }}
      </span>
    </template>
    </pv-card>
  </div>
</section>
</template>

<script>
/**
 * @summary Grid view displaying meal cards with thumbnail and tit
 * @author Juan Sebastian Estupiñan
 */
export default {
  name: 'MealList',
  props: {
    meals: {
      type: Array,
      required: true
    }
  }
}

```

```
};
</script>
```

7. Ensamblaje en src/App.vue

Integra las dos columnas mediante el sistema de rejilla de PrimeFlex, consumiendo el servicio de la API:

```
<template>
  <div class="min-h-screen flex flex-column surface-ground">
    <the-toolbar />

    <main role="main" class="p-4 flex-grow-1">
      <div class="mb-4">
        <h1 class="text-3xl font-bold text-900 mb-1">{{ ${'title'
        <p class="text-600 text-lg m-0">{{ ${'description' } }}</f
      </div>

      <div class="grid">
        <!-- Panel Izquierdo: Categorías -->
        <div class="col-12 md:col-4 lg:col-3">
          <pv-panel :header='${'categories'}' class="shadow-1">
            <category-list
              :categories="categories"
              :selected-category="selectedCategory"
              @select-category="fetchMeals"
            />
          </pv-panel>
        </div>

        <!-- Panel Derecho: Platos -->
        <div class="col-12 md:col-8 lg:col-9">
          <pv-panel :header="selectedCategory ? `${${'meals'}}` -
            <meal-list :meals="meals" />
          </pv-panel>
        </div>
      </div>
    </main>

    <the-footer />
```

```
</div>
</template>

<script>
/**
 * @summary Root component orchestrating layout panels, service ir
 * @author Juan Sebastian Estupiñan
 */
import { ref, onMounted } from 'vue';
import TheToolbar from './public/components/the-toolbar.component.vue';
import TheFooter from './public/components/the-footer.component.vue';
import CategoryList from './meal/components/category-list.component.vue';
import MealList from './meal/components/meal-list.component.vue';
import { MealApiService } from './meal/services/meal-api.service';

export default {
  name: 'App',
  components: {
    TheToolbar,
    TheFooter,
    CategoryList,
    MealList
  },
  setup() {
    const apiService = new MealApiService();
    const categories = ref([]);
    const meals = ref([]);
    const selectedCategory = ref('');

    const loadCategories = async () => {
      try {
        categories.value = await apiService.getCategories();
        if (categories.value.length > 0) {
          await fetchMeals(categories.value[0].name);
        }
      } catch (error) {
        console.error('Failed to load categories:', error);
      }
    };
  }
};
```

```
const fetchMeals = async (categoryName) => {
  try {
    selectedCategory.value = categoryName;
    meals.value = await apiService.getMealsByCategory(categoryName)
  } catch (error) {
    console.error('Failed to load meals for ${categoryName}', error);
  }
};

onMounted(() => {
  loadCategories();
});

return {
  categories,
  meals,
  selectedCategory,
  fetchMeals
};
}
};
</script>
```

8. Documentación README.md

Crea el archivo README.md en inglés con la estructura solicitada:

```
# MealAll - Web Application

## Description
MealAll is a Vue 3 single-page application built with Vite that di

## Features
- Domain-Driven Design (DDD) layered architecture ('public' and 'n
- Internationalization (i18n) support with English as default and
- Two-panel responsive layout with category navigation and meal c
- Design patterns: Request/Response, Resource, and Assembler.
- Accessibility support using standard WAI-ARIA attributes.
```

```
## Dependencias
- Vue 3
- Vite
- Axios
- Vue-118n
- PrimeVue (Material Theme)
- PrimeFlex
- PrimeIcons
```

```
## Author
- Name: Juan Sebastian Estupiñan
- Course: ASI0730 - Web Applications
```

9. Lista de Verificación para Entrega

1. **Compilación limpia:** Ejecuta `npm run build` para comprobar que no existan errores de sintaxis o importación.
2. **Nombres de archivo:** Verifica que cada archivo mantenga el sufijo requerido (`.component.vue`, `.service.js`, `.entity.js`, `.assembler.js`).
3. **Template tags:** Asegúrate de que las etiquetas en los templates excluyan la palabra `component` (ej. `<the-toolbar>`, `<category-list>`, `<pv-card>`).
4. **Empaquetado:** Borra la carpeta `node_modules` y dist antes de comprimir. Empaqueta la raíz con el formato `pc1nrcucode.zip` (por ejemplo: `pc14381u202212345.zip`).